

МРС-64 — Реализовать деактивацию товаров, исчезнувших из каталога Varus

Summary

Добавить управляемую деактивацию записей `product_catalog`, которые перестали обнаруживаться во время полного обновления каталога.

После выполнения задачи `catalog-refresh` должен:

1. создать отдельную сессию обновления каталога;
2. пометить все обнаруженные товары текущей сессией;
3. после успешного полного discovery определить активные товары, не встреченные в текущей сессии;
4. деактивировать только товары, отсутствующие в течение настроенного периода;
5. не удалять строки физически;
6. сохранять историю цен и связи с `product`;
7. автоматически реактивировать товар, если он снова появился;
8. не выполнять деактивацию при неполном или ошибочном discovery.

Главная цель задачи:

`product_catalog` должен отражать актуальный состав каталога Varus, не удаляя исторические данные.

Context

Репозиторий:

`https://github.com/bodomus/PriceCrawler`

Базовая ветка:

`MPC-63`

Создать отдельную рабочую ветку:

`Codex/MPC-64`

Dependencies

Задача зависит от завершённых:

```
MPC-61 – постоянный product_catalog
MPC-62 – RefreshProductCatalogUseCase
MPC-63 – CollectProductPricesUseCase + oldest-first
```

До начала реализации убедиться, что в базовой ветке существуют:

- таблица `product_catalog` ;
- поля:
 - `first_discovered_at` ;
 - `last_discovered_at` ;
 - `is_active` ;
 - `last_checked_at` ;
 - `next_check_at` ;
 - `consecutive_errors` ;
- batch upsert обнаруженных URL;
- `RefreshProductCatalogUseCase` ;
- Worker-команда:

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

- `CollectProductPricesUseCase` ;
- выбор только активных товаров в oldest-first flow;
- автоматическая реактивация записи при повторном discovery.

Если перечисленные зависимости отсутствуют, не создавать параллельную реализацию.
Зафиксировать блокирующую зависимость.

Problem

Сейчас discovery обновляет:

```
last_discovered_at
is_active = true
```

для найденных URL.

Но товары, которые больше не встречаются в каталоге Varus, навсегда остаются:

```
is_active = true
```

Из-за этого `CollectProductPricesUseCase` продолжает выбирать:

- удалённые товары;
- устаревшие карточки;
- URL, больше не присутствующие в каталоге;
- товары, снятые с продажи;
- товары, временно или окончательно исключённые Varus.

Физическое удаление записей недопустимо, так как необходимо сохранить:

- историю цен;
- ссылки на старые queue items;
- статистику;
- возможность реактивации;
- аудит изменения каталога.

Goal

Добавить безопасный flow:

```
catalog-refresh start
  ↓
создание refresh session
  ↓
полный discovery
  ↓
upsert найденных URL с refresh_session_id
  ↓
проверка полноты discovery
  ↓
деактивация давно не обнаруженных товаров
  ↓
refresh session finish
```

Деактивация должна выполняться только после полностью успешного catalog refresh.

Required architecture

Соблюдать существующие слои:

```
Domain
Application
Infrastructure
Worker
```

Domain

Domain может содержать:

- immutable models;
- repository interfaces;
- enums;
- результаты операций.

Domain не должен зависеть от:

- PostgreSQL;
- Npgsql;
- EF Core;
- Application;
- Infrastructure;
- Worker;
- Web.

Application

Application должен содержать:

- orchestration деактивации;
- решение о том, когда деактивация допустима;
- построение результата;
- logging;
- обработку ошибок.

Application не должен содержать SQL.

Infrastructure

Infrastructure должен содержать:

- schema changes;
- PostgreSQL routines;
- repository implementation;
- mapping;
- транзакционные операции.

Worker

Worker не должен содержать бизнес-логику деактивации.

Scope

1. Добавить сущность catalog refresh session

Добавить таблицу:

product_catalog_refresh

Предлагаемая структура:

```
create table if not exists product_catalog_refresh
(
    id                bigint generated always as identity primary key,
    source            varchar(50) not null,
    discovery_source   varchar(50) not null,
    started_at        timestampz not null,
    finished_at       timestampz,
    status            varchar(20) not null,
    discovered_count   integer not null default 0,
    accepted_count     integer not null default 0,
    inserted_count     integer not null default 0,
    updated_count      integer not null default 0,
    deactivated_count  integer not null default 0,
    reactivated_count  integer not null default 0,
    error_code        varchar(100),
    error_message      varchar(1000),
    created_at         timestampz not null default now(),
    updated_at         timestampz not null default now()
);
```

Допускается другое имя таблицы, если оно лучше соответствует текущему стилю, но семантика должна сохраниться.

Статусы

Допустимые значения:

running
ok
error
cancelled

Добавить check constraint.

2. Связать `product_catalog` с `refresh session`

Добавить в `product_catalog`:

```
last_seen_refresh_id bigint null
```

Foreign key:

```
references product_catalog_refresh(id)
```

Политика удаления:

```
on delete restrict
```

Добавить индекс:

```
create index if not exists ix_product_catalog_last_seen_refresh_id  
on product_catalog(last_seen_refresh_id);
```

Поведение:

- каждый товар, обнаруженный в текущем refresh, получает `last_seen_refresh_id = currentRefreshId`;
- существующий `last_discovered_at` продолжает обновляться;
- новые товары получают текущий refresh ID;
- реактивированные товары получают текущий refresh ID.

3. Добавить даты деактивации и реактивации

Добавить в `product_catalog`:

```
deactivated_at timestampz null,  
reactivated_at timestampz null
```

При деактивации

```
is_active = false  
deactivated_at = now  
next_check_at = null  
reservation fields cleared  
updated_at = now
```

При повторном обнаружении

Если запись была inactive:

```
is_active = true
reactivated_at = discovered_at
deactivated_at = null
updated_at = now
```

`first_discovered_at` не изменять.

`last_checked_at` не сбрасывать.

`consecutive_errors` не сбрасывать только из-за реактивации, если нет отдельного обоснования. Предпочтительно сохранять значение до первого успешного price check.

4. Расширить catalog upsert

Изменить:

```
ProductCatalogUpsertItem
```

Добавив:

```
long RefreshId
```

или передавать refresh ID отдельным параметром batch repository method.

Предпочтительный интерфейс:

```
Task<ProductCatalogUpsertResult> UpsertDiscoveredAsync(
    long refreshId,
    IReadOnlyCollection<ProductCatalogUpsertItem> items,
    CancellationToken ct);
```

Не дублировать refresh ID в каждом элементе batch без необходимости.

DB routine должна:

- записывать `last_seen_refresh_id`;
 - возвращать количество реактивированных строк;
 - корректно различать inserted, updated и reactivated.
-

5. Расширить результат upsert

Изменить:

```
ProductCatalogUpsertResult
```

Добавив:

```
int ReactivatedCount
```

Итоговая модель:

```
public sealed record ProductCatalogUpsertResult(  
    int ReceivedCount,  
    int InsertedCount,  
    int UpdatedCount,  
    int ReactivatedCount);
```

Семантика:

- `InsertedCount` — новые строки;
- `UpdatedCount` — существующие активные строки;
- `ReactivatedCount` — существующие inactive строки, снова обнаруженные;
- одна запись не должна одновременно считаться updated и reactivated.

Refresh session repository

6. Добавить domain models

Добавить:

```
public sealed record ProductCatalogRefreshSession(  
    long Id,  
    string Source,  
    string DiscoverySource,  
    DateTimeOffset StartedAtUtc,  
    DateTimeOffset? FinishedAtUtc,  
    string Status,  
    int DiscoveredCount,  
    int AcceptedCount,  
    int InsertedCount,  
    int UpdatedCount,  
    int DeactivatedCount,  
    int ReactivatedCount,
```



```
string? ErrorCode,  
string? ErrorMessage);
```

Добавить result model старта или использовать `long`.

7. Добавить repository interface

Добавить:

```
public interface IProductCatalogRefreshRepository  
{  
    Task<long> StartAsync(  
        string source,  
        string discoverySource,  
        DateTimeOffset startedAtUtc,  
        CancellationToken ct);  
  
    Task CompleteAsync(  
        long refreshId,  
        ProductCatalogRefreshCompletion completion,  
        CancellationToken ct);  
  
    Task FailAsync(  
        long refreshId,  
        string status,  
        string errorCode,  
        string? errorMessage,  
        DateTimeOffset finishedAtUtc,  
        CancellationToken ct);  
  
    Task<ProductCatalogRefreshSession?> GetByIdAsync(  
        long refreshId,  
        CancellationToken ct);  
}
```

Добавить immutable model:

```
public sealed record ProductCatalogRefreshCompletion(  
    int DiscoveredCount,  
    int AcceptedCount,  
    int InsertedCount,  
    int UpdatedCount,  
    int DeactivatedCount,  
    int ReactivatedCount,  
    DateTimeOffset FinishedAtUtc);
```

Не помещать SQL-specific details в Domain.

Deactivation repository API

8. Расширить `IProductCatalogRepository`

Добавить:

```
Task<int> DeactivateMissingAsync(  
    string source,  
    long currentRefreshId,  
    DateTimeOffset notSeenSinceUtc,  
    DateTimeOffset deactivatedAtUtc,  
    CancellationToken ct);
```

Метод должен деактивировать только записи:

```
source совпадает  
is_active = true  
last_seen_refresh_id != currentRefreshId  
last_discovered_at < notSeenSinceUtc
```

Обязательное поведение:

- строки текущего refresh не деактивируются;
- inactive строки повторно не обновляются;
- товары другого source не затрагиваются;
- reservation очищаются;
- `next_check_at` очищается;
- физическое удаление не выполняется.

Configuration

9. Добавить настройки

Добавить в `CrawlerOptions`:

```
public int CatalogMissingGracePeriodDays { get; set; } = 14;  
public int CatalogMinimumExpectedUrls { get; set; } = 1000;  
public double CatalogMinimumPreviousRatio { get; set; } = 0.5d;  
public bool CatalogDeactivationEnabled { get; set; } = true;
```

Пример конфигурации:

```
{
  "Crawler": {
    "CatalogDeactivationEnabled": true,
    "CatalogMissingGracePeriodDays": 14,
    "CatalogMinimumExpectedUrls": 1000,
    "CatalogMinimumPreviousRatio": 0.5
  }
}
```

10. Семантика настроек

CatalogMissingGracePeriodDays

Товар деактивируется, только если:

```
last_discovered_at < now - grace period
```

Минимальное безопасное значение:

1

Значения меньше 1 нормализовать до 1.

CatalogMinimumExpectedUrls

Если discovery вернул меньше этого количества, деактивацию не выполнять.

Это защита от:

- сломанного HTML parser;
- частичного сетевого сбоя;
- пустого seed file;
- блокировки Varus;
- изменения структуры сайта.

Минимальное значение:

1

CatalogMinimumPreviousRatio

Сравнивать текущий accepted count с числом активных товаров перед refresh.

Пример:

```
active before refresh = 40000
ratio = 0.5
minimum safe accepted = 20000
```

Если текущий accepted count меньше порога, деактивацию не выполнять.

Допустимый диапазон:

```
0.0 < value <= 1.0
```

Некорректные значения должны приводиться к безопасному default:

```
0.5
```

`CatalogDeactivationEnabled`

При `false`:

- refresh выполняется;
- upsert выполняется;
- реактивация работает;
- деактивация не выполняется;
- run завершается успешно;
- в note/log указывается:

```
deactivation_disabled
```

Safety requirements

11. Деактивация разрешена только после полного успеха discovery

Запрещено вызывать `DeactivateMissingAsync`, если:

- discovery бросил исключение;
 - discovery был отменён;
 - upsert завершился ошибкой;
 - accepted count равен 0;
 - discovery result не прошёл safety thresholds;
 - refresh session уже имеет error/cancelled status;
 - source не определён;
 - current refresh ID отсутствует.
-

12. Safety threshold по абсолютному числу

Условие:

```
acceptedCount >= CatalogMinimumExpectedUrls
```

Если условие не выполнено:

```
refresh status = error  
error code = catalog_refresh_below_minimum  
deactivation count = 0
```

Upsert уже найденных URL допускается сохранить, но деактивацию не выполнять.

Это должно быть явно отражено в result и логах.

13. Safety threshold относительно предыдущего каталога

Перед discovery или до деактивации получить число активных записей:

```
Task<int> GetActiveCountAsync(  
    string source,  
    CancellationToken ct);
```

Условие:

```
acceptedCount >= ceil(activeCountBefore * CatalogMinimumPreviousRatio)
```

Если активный каталог пуст:

```
ratio check не применяется
```

Если условие не выполнено:

```
refresh status = error  
error code = catalog_refresh_ratio_too_low  
deactivation count = 0
```

14. Полный refresh и фильтры

Если настроен:

```
"VegetablesUrlContains": "/ovochi"
```

это не является полным refresh всего каталога.

Поэтому деактивация должна быть запрещена при непустом `VegetablesUrlContains`.

Required behavior:

```
filter non-empty
→ upsert discovered subset allowed
→ deactivation skipped
→ refresh may finish ok
→ reason = scoped_filter_active
```

Нельзя деактивировать весь каталог на основании частичного овощного discovery.

15. Discovery mode

Деактивация допустима только для discovery mode, который считается полным источником каталога.

Для текущей версии:

```
CategorySeeds
```

может считаться полным только при:

- пустом `VegetablesUrlContains`;
- успешно обработанном seed-файле;
- отсутствии fatal discovery errors;
- прохождении safety thresholds.

Для `Api` и `Sitemap` деактивацию не включать автоматически, пока нет подтверждённой полноты этих источников.

Добавить явное определение:

```
bool IsFullCatalogDiscovery(...)
```

Не определять полноту по строковым сравнениям в нескольких местах.

RefreshProductCatalogUseCase changes

16. Обновить execution flow

Новый порядок:

```
crawler_run_start
→ determine discovery mode
→ product_catalog_refresh_start
→ get active count before
→ discovery
→ batch upsert with refresh id
→ validate safety thresholds
→ deactivate missing
→ product_catalog_refresh_complete
→ crawler_run_finish
```

17. Порядок старта refresh session

Так как фактический discovery source может быть известен только после выполнения strategy, допускаются два варианта.

Предпочтительный вариант

Получить strategy metadata без запуска discovery:

```
source = varus
discovery_source = category-seed
```

и создать session до сетевого обхода.

Допустимый вариант

Создать session с:

```
discovery_source = pending
```

и обновить после определения source.

Но запись refresh session должна существовать во время длительного обхода категорий.

18. Result model

Расширить:

```
RefreshProductCatalogResult
```

Добавив:

```
long RefreshId
int ReactivatedCount
int DeactivatedCount
bool DeactivationExecuted
string? DeactivationSkipReason
```

Предлагаемый результат:

```
public sealed record RefreshProductCatalogResult(
    long RunId,
    long RefreshId,
    string Status,
    string Source,
    int DiscoveredCount,
    int AcceptedCount,
    int InsertedCount,
    int UpdatedCount,
    int ReactivatedCount,
    int DeactivatedCount,
    int SkippedCount,
    bool DeactivationExecuted,
    string? DeactivationSkipReason,
    string? ErrorCode,
    string? Message);
```

Deactivation behavior

19. SQL semantics

Пример required routine:

```
product_catalog_deactivate_missing
```

Псевдологика:


```

update product_catalog
set
    is_active = false,
    deactivated_at = p_deactivated_at,
    next_check_at = null,
    reserved_at = null,
    reserved_until = null,
    reserved_by = null,
    updated_at = now()
where source = p_source
    and is_active = true
    and coalesce(last_seen_refresh_id, 0) <> p_current_refresh_id
    and last_discovered_at < p_not_seen_since;

```

Routine должна вернуть количество изменённых строк.

20. Grace period

Пример:

```

refresh date: 2026-06-05
grace period: 14 days
cutoff: 2026-05-22

```

Товар:

```
last_discovered_at = 2026-06-01
```

не деактивируется, даже если отсутствовал в текущем refresh.

Товар:

```
last_discovered_at = 2026-05-01
```

деактивируется, если не встречен в текущем refresh.

Это защищает от временно отсутствующих категорий и кратковременных сбоев.

21. Reactivation

Если inactive запись снова обнаружена:

```
is_active = true
deactivated_at = null
reactivated_at = discoveredAt
last_seen_refresh_id = currentRefreshId
last_discovered_at = discoveredAt
```

Она должна снова стать доступной для `CollectProductPricesUseCase`.

Для ускорения первой проверки после реактивации:

```
next_check_at = null
```

Допускается выставить `next_check_at = null`, чтобы товар попал в oldest-first selection.

`last_checked_at` сохраняется для истории.

Interaction with CollectProductPricesUseCase

22. Inactive products

Убедиться, что oldest-first routine из MPC-63 продолжает использовать:

```
where is_active = true
```

Inactive товар:

- не выбирается;
- не резервируется;
- не enqueue;
- не получает новые snapshots.

23. Active lease during deactivation

Если товар имеет активную reservation, но подлежит деактивации:

Предпочтительно:

- не деактивировать запись с неистёкшим lease;
- оставить её до следующего refresh.

Условие:

```
and (  
    reserved_until is null  
    or reserved_until <= p_deactivated_at  
)
```

Это снижает риск гонки между `catalog-refresh` и `collect-prices`.

Если выбран другой подход, он должен быть транзакционно безопасным и документирован.

24. Concurrent catalog refresh

Два `catalog-refresh` одного source не должны одновременно выполнять деактивацию.

Добавить PostgreSQL advisory lock либо эквивалентный механизм.

Предпочтительно:

```
pg_try_advisory_lock(hash(source + ":catalog-refresh"))
```

Поведение второго запуска:

```
status = error  
error code = catalog_refresh_already_running
```

Не ждать lock бесконечно.

Lock должен освобождаться:

- после успеха;
- после ошибки;
- после cancellation;
- при закрытии соединения.

Допускается уникальный partial index на running session, если он надёжно предотвращает конкуренцию.

Error handling

25. Discovery error

При discovery error:

- refresh session → `error`;

- crawler run → `error` ;
 - deactivation не выполняется;
 - существующий `product_catalog` не меняет active flags;
 - exception логируется;
 - result содержит stable error code.
-

26. Upsert error

При upsert error:

- refresh session → `error` ;
 - deactivation не выполняется;
 - crawler run → `error` ;
 - частично записанные batch changes не должны приводить к массовой деактивации.
-

27. Deactivation error

Если upsert успешен, но деактивация упала:

- refresh session → `error` ;
- crawler run → `error` ;
- error code:

catalog_deactivation_failed

- не возвращать ложный success.
-

28. Cancellation

При cancellation:

- refresh session → `cancelled` , если возможно;
- crawler run → `error` или существующий поддерживаемый cancelled status;
- деактивация не выполняется;
- `OperationCanceledException` повторно выбрасывается.

Не превращать cancellation в обычный result `ok` .

DB routines

Добавить либо изменить:

```
product_catalog_refresh_start  
product_catalog_refresh_complete  
product_catalog_refresh_fail  
product_catalog_refresh_get_by_id  
product_catalog_get_active_count  
product_catalog_upsert_discovered  
product_catalog_deactivate_missing
```

Все routines должны:

- использовать parameterized calls;
- быть идемпотентно создаваемыми bootstrap mechanism;
- не использовать dynamic SQL без необходимости;
- корректно работать с UTC timestamps;
- возвращать детерминированные counters.

Repository implementations

29. Добавить PostgreSQL repository

Добавить:

```
PgProductCatalogRefreshRepository
```

Он должен использовать:

- `PgRoutineExecutor`;
- `DbRoutineCall`;
- существующий connection abstraction;
- `CancellationToken`.

Зарегистрировать:

```
services.AddScoped<  
    IProductCatalogRefreshRepository,  
    PgProductCatalogRefreshRepository>();
```

Logging

Добавить структурированные агрегированные сообщения.

Start

```
Product catalog refresh session started. RunId={RunId};  
RefreshId={RefreshId}; Source={Source}; DiscoverySource={DiscoverySource}
```

Discovery

```
Product catalog discovery completed. RunId={RunId}; RefreshId={RefreshId};  
Discovered={Discovered}
```

Safety check

```
Product catalog refresh safety check. RefreshId={RefreshId};  
ActiveBefore={ActiveBefore}; Accepted={Accepted};  
MinimumExpected={MinimumExpected}; MinimumRatio={MinimumRatio};  
IsSafe={IsSafe}
```

Deactivation skipped

```
Product catalog deactivation skipped. RefreshId={RefreshId}; Reason={Reason}
```

Completion

```
Product catalog refresh completed. RunId={RunId}; RefreshId={RefreshId};  
Inserted={Inserted}; Updated={Updated}; Reactivated={Reactivated};  
Deactivated={Deactivated}
```

Failure

```
Product catalog refresh failed. RunId={RunId}; RefreshId={RefreshId};  
ErrorCode={ErrorCode}
```

Не логировать все URL на уровне Information.

Tests

Добавить unit tests, PostgreSQL integration tests и concurrency tests.

Unit tests: safety policy

1. Deactivation enabled

CatalogRefreshSafetyPolicy_ValidFullRefresh_AllowsDeactivation

2. Feature disabled

CatalogRefreshSafetyPolicy_DeactivationDisabled_SkipsDeactivation

3. Scoped filter

CatalogRefreshSafetyPolicy_VegetablesFilterActive_SkipsDeactivation

4. Below absolute minimum

CatalogRefreshSafetyPolicy_AcceptedBelowMinimum_RejectsRefresh

5. Below previous ratio

CatalogRefreshSafetyPolicy_AcceptedBelowPreviousRatio_RejectsRefresh

6. Empty previous catalog

CatalogRefreshSafetyPolicy_NoPreviousCatalog_DoesNotApplyRatioCheck

7. Unsupported discovery mode

CatalogRefreshSafetyPolicy_NonFullDiscoveryMode_SkipsDeactivation

Unit tests: use case

8. Successful full refresh

ExecuteAsync_FullSafeRefresh_UpsertsAndDeactivatesMissing

Проверить порядок:

```
crawler start  
refresh session start  
active count  
discovery  
upsert  
safety check  
deactivate  
refresh complete  
crawler finish
```

9. Discovery error

```
ExecuteAsync_DiscoveryFails_DoesNotDeactivate
```

10. Upsert error

```
ExecuteAsync_UpsertFails_DoesNotDeactivate
```

11. Safety failure

```
ExecuteAsync_SafetyThresholdFails_DoesNotDeactivateAndFinishesError
```

12. Filtered refresh

```
ExecuteAsync_ScopedFilterActive_UpsertsButDoesNotDeactivate
```

13. Feature disabled

```
ExecuteAsync_DeactivationDisabled_UpsertsWithoutDeactivation
```

14. Reactivation count

```
ExecuteAsync_ReactivatedItems_ReturnsReactivatedCount
```

15. Cancellation

```
ExecuteAsync_Cancelled_DoesNotDeactivateAndMarksSessionCancelled
```


16. Deactivation error

```
ExecuteAsync_DeactivationFails_FinishesRefreshWithError
```

PostgreSQL integration tests

17. Refresh session insert

```
ProductCatalogRefresh_Start_CreatesRunningSession
```

18. Refresh session complete

```
ProductCatalogRefresh_Complete_StoresAllCounters
```

19. Refresh session fail

```
ProductCatalogRefresh_Fail_StoresStableErrorData
```

20. Upsert marks refresh ID

```
UpsertDiscoveredAsync_SetsLastSeenRefreshId
```

21. Missing item deactivation

Создать:

```
A – seen in current refresh  
B – not seen, older than grace period  
C – not seen, newer than grace period  
D – already inactive  
E – another source
```

Ожидается:

```
A active  
B inactive  
C active
```

D inactive unchanged
E unchanged

Тест:

DeactivateMissingAsync_DeactivatesOnlyEligibleRows

22. Deactivation clears scheduling state

DeactivateMissingAsync_ClearsNextCheckAndExpiredReservation

23. Active lease protected

DeactivateMissingAsync_ActiveLease_IsNotDeactivated

24. Reactivation

UpsertDiscoveredAsync_InactiveItem_ReactivatesAndClearsDeactivatedAt

Проверить:

- `is_active = true`;
- `reactivated_at` заполнено;
- `deactivated_at = null`;
- `next_check_at = null`;
- строка не дублируется.

25. History preserved

DeactivateMissingAsync_DoesNotDeleteProductOrPriceHistory

Проверить сохранность:

- product catalog row;
- product;
- price snapshots;
- queue history.

26. Concurrent refresh lock

CatalogRefresh_ConcurrentRuns_OnlyOneCanProceed

Второй запуск должен получить:

catalog_refresh_already_running

27. Schema idempotency

Повторный schema bootstrap не должен падать.

End-to-end tests

28. Full refresh and deactivation

Сценарий:

1. первый refresh находит A, B, C;
2. второй refresh находит A, C;
3. grace period для B искусственно истёк;
4. второй refresh завершается успешно;
5. B становится inactive.

Тест:

RefreshProductCatalogUseCase_MissingOldItem_DeactivatesAfterSuccessfulRefresh

29. Failed refresh preserves active products

Сценарий:

1. каталог содержит A, B, C;
2. discovery второго refresh падает;
3. ни одна запись не деактивируется.

Тест:

RefreshProductCatalogUseCase_FailedDiscovery_DoesNotDeactivateCatalog

30. Partial suspicious refresh preserves active products

Сценарий:

1. active count = 1000;
2. ratio threshold = 0.5;
3. discovery возвращает 100;
4. refresh завершается error;
5. деактивация не выполняется.

31. Reactivated item becomes due

Сценарий:

1. товар inactive;
2. следующий refresh снова обнаруживает его;
3. товар active;
4. `next_check_at = null`;
5. `CollectProductPricesUseCase` может выбрать его.

Documentation

Обновить:

docs/architecture.md
README.md

Описать lifecycle

```
discovered
→ active
→ missing during refresh
→ grace period
→ inactive
→ discovered again
→ reactivated
```

Описать разницу

```
is_active = false
```

не означает физическое удаление

Описать safety guards

- minimum expected URL count;
- ratio to previous active catalog;
- no deactivation with scoped filter;
- no deactivation after failed discovery;
- no deactivation for unsupported incomplete discovery modes;
- single refresh lock.

Добавить пример конфигурации

```
{
  "Crawler": {
    "CatalogDeactivationEnabled": true,
    "CatalogMissingGracePeriodDays": 14,
    "CatalogMinimumExpectedUrls": 1000,
    "CatalogMinimumPreviousRatio": 0.5
  }
}
```

Добавить SQL-проверки

```
select
  is_active,
  count(*)
from product_catalog
group by is_active;
```

```
select
  id,
  normalized_url,
  last_discovered_at,
  last_seen_refresh_id,
  is_active,
  deactivated_at,
  reactivated_at
from product_catalog
order by updated_at desc
limit 100;
```

```
select *  
from product_catalog_refresh  
order by id desc  
limit 20;
```

Non-goals

В рамках MPC-64 запрещено реализовывать:

- физическое удаление товаров;
- удаление истории цен;
- удаление queue history;
- UI управления inactive товарами;
- Web API каталога;
- ручное включение/выключение отдельных товаров;
- категорийные приоритеты;
- разные grace periods по категориям;
- динамическую частоту проверок;
- автоматический cron;
- cloud scheduler;
- очистку старых refresh sessions;
- multi-source ATB support;
- изменение HTML parser;
- новый discovery strategy;
- переход на streaming discovery;
- Kafka;
- RabbitMQ;
- Redis;
- distributed catalog service;
- изменение ценовой логики;
- изменение структуры `product` без прямой необходимости.

Important design decisions

1. Только soft deactivation

Записи `product_catalog` не удаляются.

2. Deactivation выполняется только после полного успешного refresh

Ошибка discovery не должна превращаться в массовую деактивацию.

3. Grace period обязателен

Отсутствие товара в одном refresh недостаточно для немедленной деактивации.

4. Safety thresholds обязательны

Слишком маленький результат discovery считается подозрительным.

5. Scoped filter запрещает глобальную деактивацию

Овощной или иной частичный фильтр не может использоваться для определения отсутствующих товаров всего каталога.

6. Reactivation автоматическая

Повторный discovery возвращает товар в active state.

7. История сохраняется

Inactive товар остаётся доступным для анализа исторических цен.

8. Два refresh одного source не работают одновременно

Необходимо исключить гонки между session IDs и массовой деактивацией.

Acceptance criteria

Задача считается выполненной, если:

- [] добавлена таблица `product_catalog_refresh`;
- [] добавлена refresh session domain model;
- [] добавлен `IProductCatalogRefreshRepository`;
- [] реализован `PgProductCatalogRefreshRepository`;
- [] refresh repository зарегистрирован в DI;
- [] `product_catalog` содержит `last_seen_refresh_id`;
- [] добавлены `deactivated_at` и `reactivated_at`;
- [] batch upsert принимает refresh ID;
- [] upsert устанавливает `last_seen_refresh_id`;
- [] upsert корректно считает reactivated items;
- [] добавлен `DeactivateMissingAsync`;
- [] деактивация является soft operation;
- [] `first_discovered_at` сохраняется;
- [] история price snapshots сохраняется;
- [] inactive товары не выбираются price collection;
- [] повторный discovery реактивирует товар;
- [] при реактивации `next_check_at = null`;

- [] grace period применяется;
- [] абсолютный safety threshold применяется;
- [] relative ratio threshold применяется;
- [] partial filtered refresh не выполняет глобальную деактивацию;
- [] failed discovery не выполняет деактивацию;
- [] failed upsert не выполняет деактивацию;
- [] deactivation failure завершает refresh ошибкой;
- [] concurrent refresh защищён lock-механизмом;
- [] active leased items не деактивируются;
- [] RefreshProductCatalogResult содержит refresh и deactivation counters;
- [] добавлены unit tests;
- [] добавлены PostgreSQL integration tests;
- [] добавлены concurrency tests;
- [] добавлены end-to-end tests;
- [] schema bootstrap идиempотентен;
- [] документация обновлена;
- [] существующие тесты проходят;
- [] solution собирается без новых warnings;
- [] unrelated code не рефакторился.

Required verification

Codex обязан выполнить:

```
dotnet restore
dotnet build --no-restore
dotnet test --no-build
```

Запустить PostgreSQL integration infrastructure проекта.

Manual verification scenario

Шаг 1. Выполнить первый полный refresh

Конфигурация:

```
{
  "Crawler": {
    "VegetablesUrlContains": "",
    "CatalogDeactivationEnabled": true,
    "CatalogMissingGracePeriodDays": 14,
    "CatalogMinimumExpectedUrls": 1000,
    "CatalogMinimumPreviousRatio": 0.5
  }
}
```



```
}  
}
```

Запуск:

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Проверить:

```
select *  
from product_catalog_refresh  
order by id desc  
limit 1;
```

Шаг 2. Проверить active catalog

```
select count(*)  
from product_catalog  
where is_active = true;
```

Шаг 3. Подготовить контролируемую missing запись

Только в dev/test БД выбрать одну запись и установить:

```
update product_catalog  
set  
    last_discovered_at = now() - interval '30 days',  
    last_seen_refresh_id = null,  
    is_active = true  
where id = :test_catalog_id;
```

Шаг 4. Выполнить следующий refresh

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Проверить:

```
select
    id,
    is_active,
    last_discovered_at,
    last_seen_refresh_id,
    deactivated_at
from product_catalog
where id = :test_catalog_id;
```

Ожидается:

```
is_active = false
deactivated_at is not null
```

Только если URL действительно не был найден в новом refresh.

Шаг 5. Проверить защиту partial refresh

Установить:

```
"VegetablesUrlContains": "/ovochi"
```

Запустить:

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Ожидается:

```
upsert subset may succeed
deactivation executed = false
skip reason = scoped_filter_active
```

Количество inactive записей не должно массово увеличиться.

Шаг 6. Проверить suspicious small result

В test environment установить:

```
"CatalogMinimumExpectedUrls": 1000000
```

Запустить refresh.

Ожидается:

```
status = error
error code = catalog_refresh_below_minimum
deactivated count = 0
```

Codex final response format

В финальном ответе Codex обязан предоставить следующие разделы.

1. Summary

Кратко описать lifecycle active/inactive/reactivated.

2. Changed files

Перечислить все изменённые и добавленные файлы с назначением.

3. Runtime flow

Показать фактический порядок:

```
crawler_run_start
→ refresh_session_start
→ active_count
→ discovery
→ catalog_upsert
→ safety_check
→ deactivate_missing
→ refresh_session_complete
→ crawler_run_finish
```

4. Safety guards

Явно описать:

- grace period;
- minimum expected count;
- previous active ratio;
- scoped filter protection;
- supported full discovery modes;
- concurrent refresh lock.

5. Database changes

Перечислить:

- новую таблицу;
- новые колонки;
- indexes;
- foreign keys;
- routines;
- изменённые routines.

6. Reactivation behavior

Подтвердить:

- inactive item становится active;
- строка не дублируется;
- `deactivated_at` очищается;
- `reactivated_at` заполняется;
- `next_check_at = null`.

7. Tests

Указать:

- unit tests;
- PostgreSQL integration tests;
- concurrency tests;
- end-to-end tests;
- фактический результат `dotnet build`;
- фактический результат `dotnet test`.

8. Manual verification evidence

Привести фактические значения:

```
RunId:
RefreshId:
DiscoverySource:
Discovered:
Accepted:
Inserted:
Updated:
Reactivated:
Deactivated:
DeactivationExecuted:
DeactivationSkipReason:
```

Active catalog before:

Active catalog after:

9. Risks and follow-up

Явно указать:

- scheduler ещё не реализован;
- cleanup старых refresh sessions ещё не реализован;
- grace period общий для всех категорий;
- полнота Api/Sitemap discovery пока не считается подтверждённой;
- физическое удаление товаров не выполняется.

Не заявлять об успешном тестировании или ручной проверке, если команды фактически не запускались.